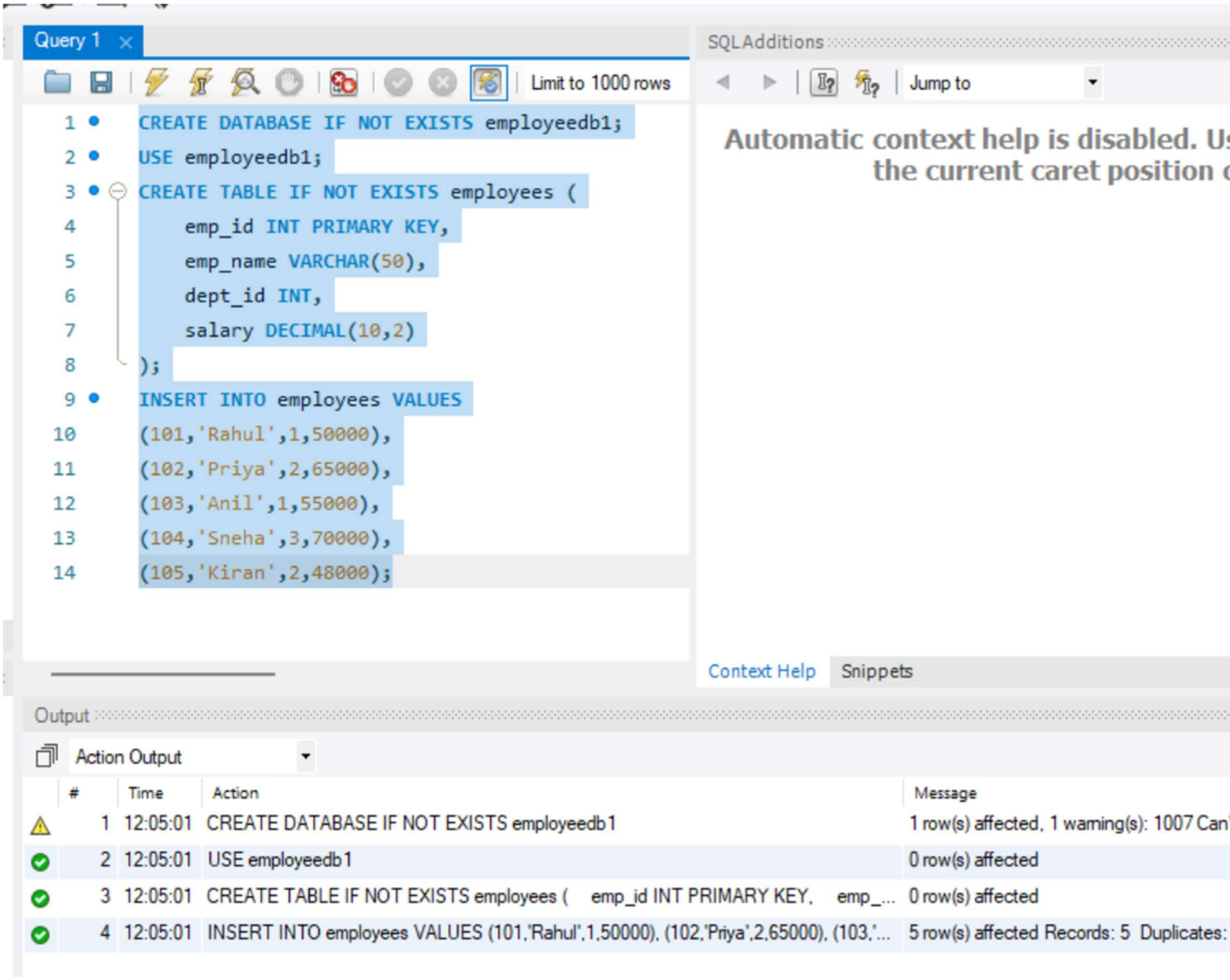
MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

1. Setup: Employees Table

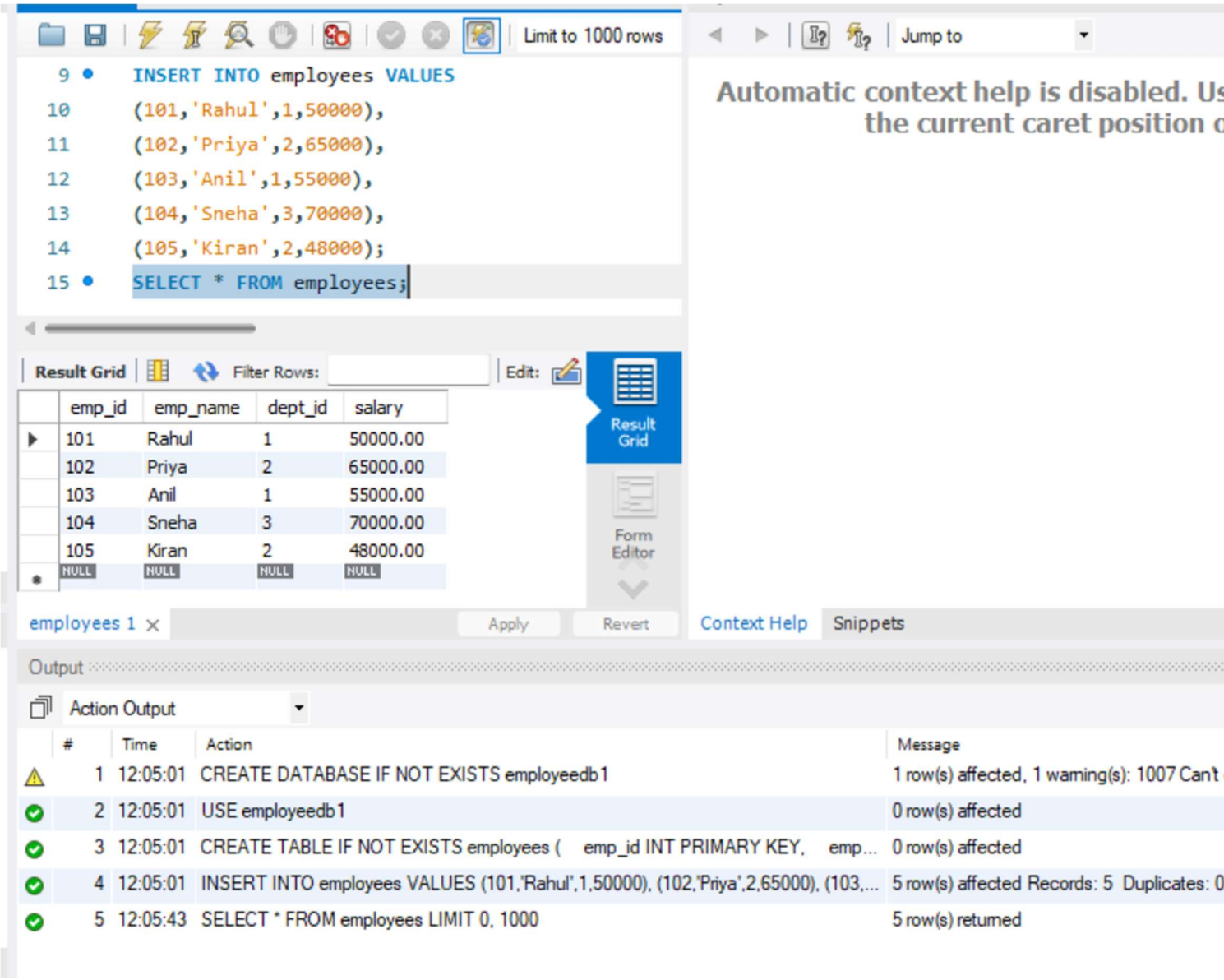
```
CREATE DATABASE IF NOT EXISTS employeedb1;
USE employeedb1;

CREATE TABLE IF NOT EXISTS employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   dept_id INT,
   salary DECIMAL(10,2)
);

INSERT INTO employees VALUES
(101,'Rahul',1,50000),
(102,'Priya',2,65000),
(103,'Anil',1,55000),
(104,'Sneha',3,70000),
(105,'Kiran',2,48000);
```



2. Verify Data

Question: Display all employee records.

SELECT * FROM employees;

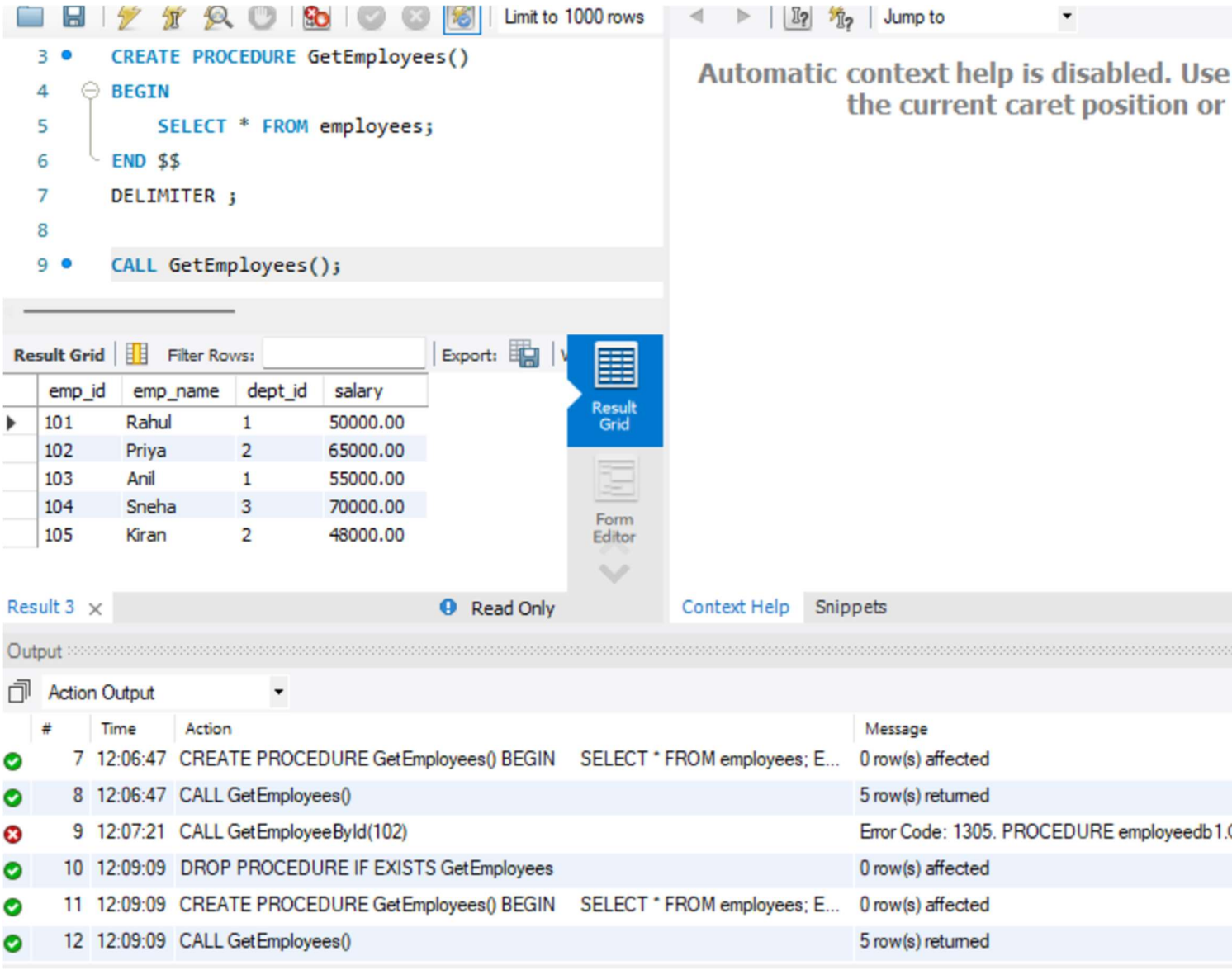


Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees.

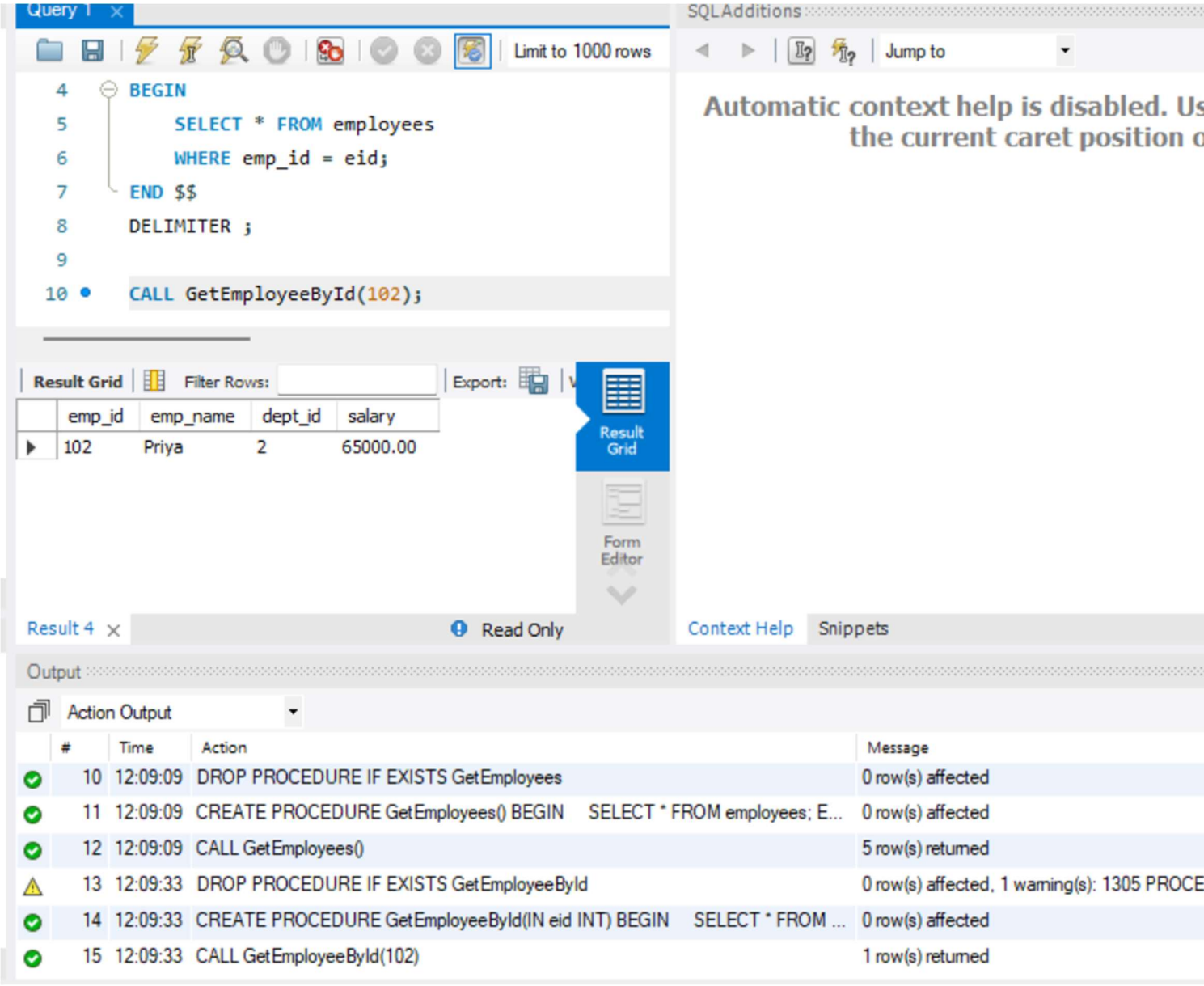
```
DROP PROCEDURE IF EXISTS GetEmployees;
DELIMITER $$

CREATE PROCEDURE GetEmployees()
BEGIN
   SELECT * FROM employees;
END $$

DELIMITER ;
```

CALL GetEmployees();



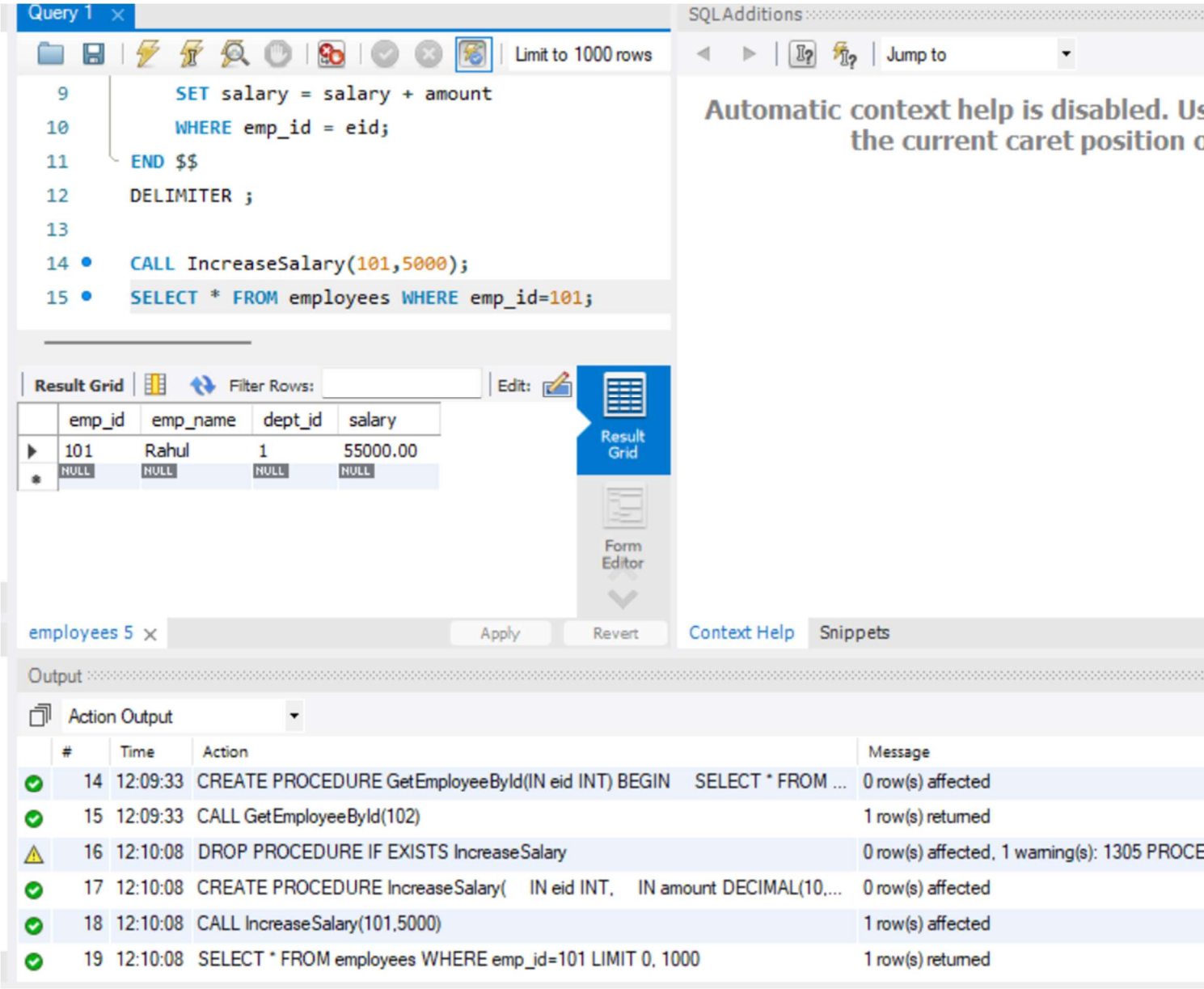
Procedure 2 – IN Parameter

Question: Create a procedure that accepts an employee ID and displays that employee's details.



Procedure 3 – Update Salary

Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.

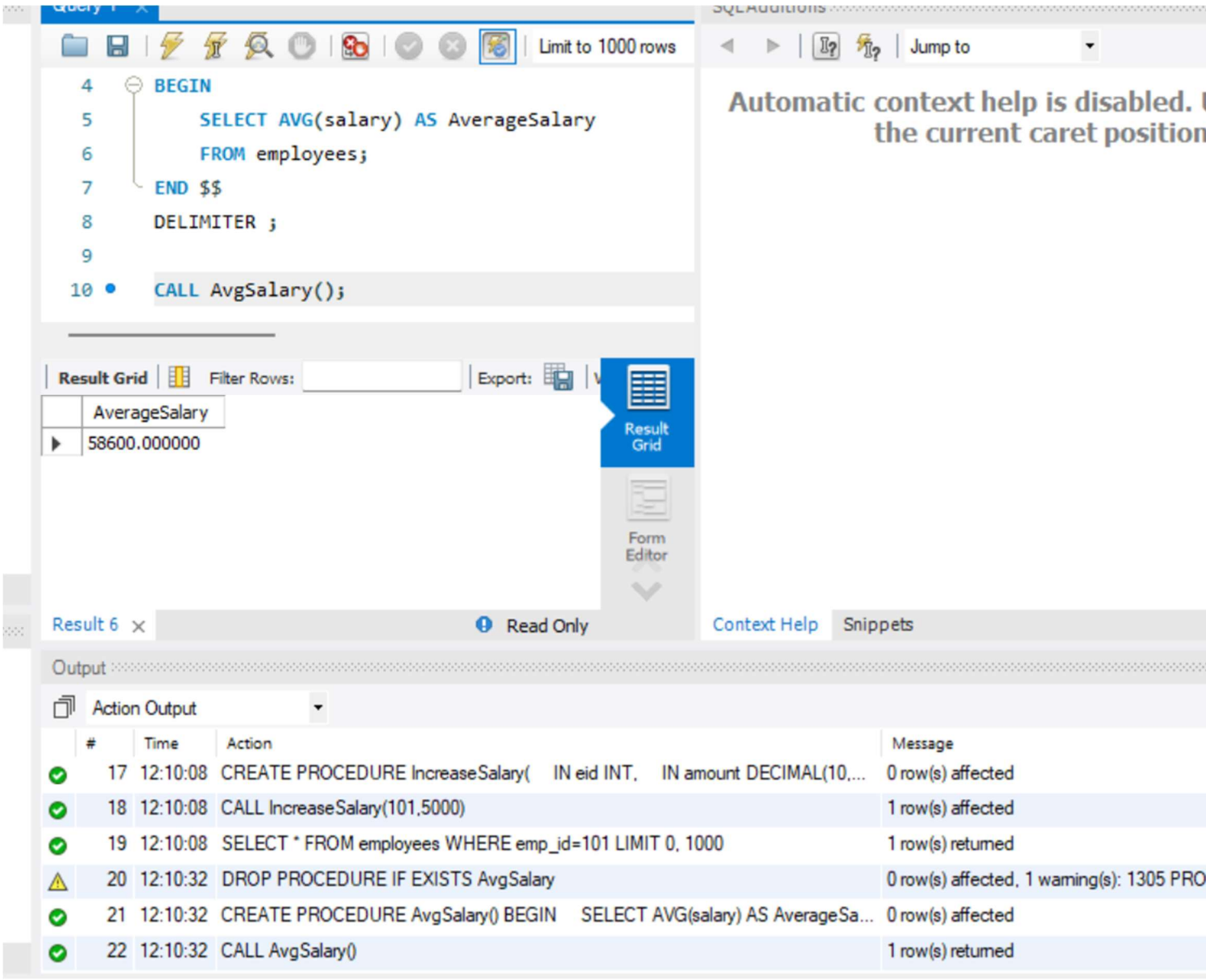


Procedure 4 – Aggregate Function

Question: Create a procedure to calculate the average salary of all employees.

```
DROP PROCEDURE IF EXISTS AvgSalary;
DELIMITER $$

CREATE PROCEDURE AvgSalary()
BEGIN
   SELECT AVG(salary) AS AverageSalary
   FROM employees;
END $$

DELIMITER ;
```

CALL AvgSalary();

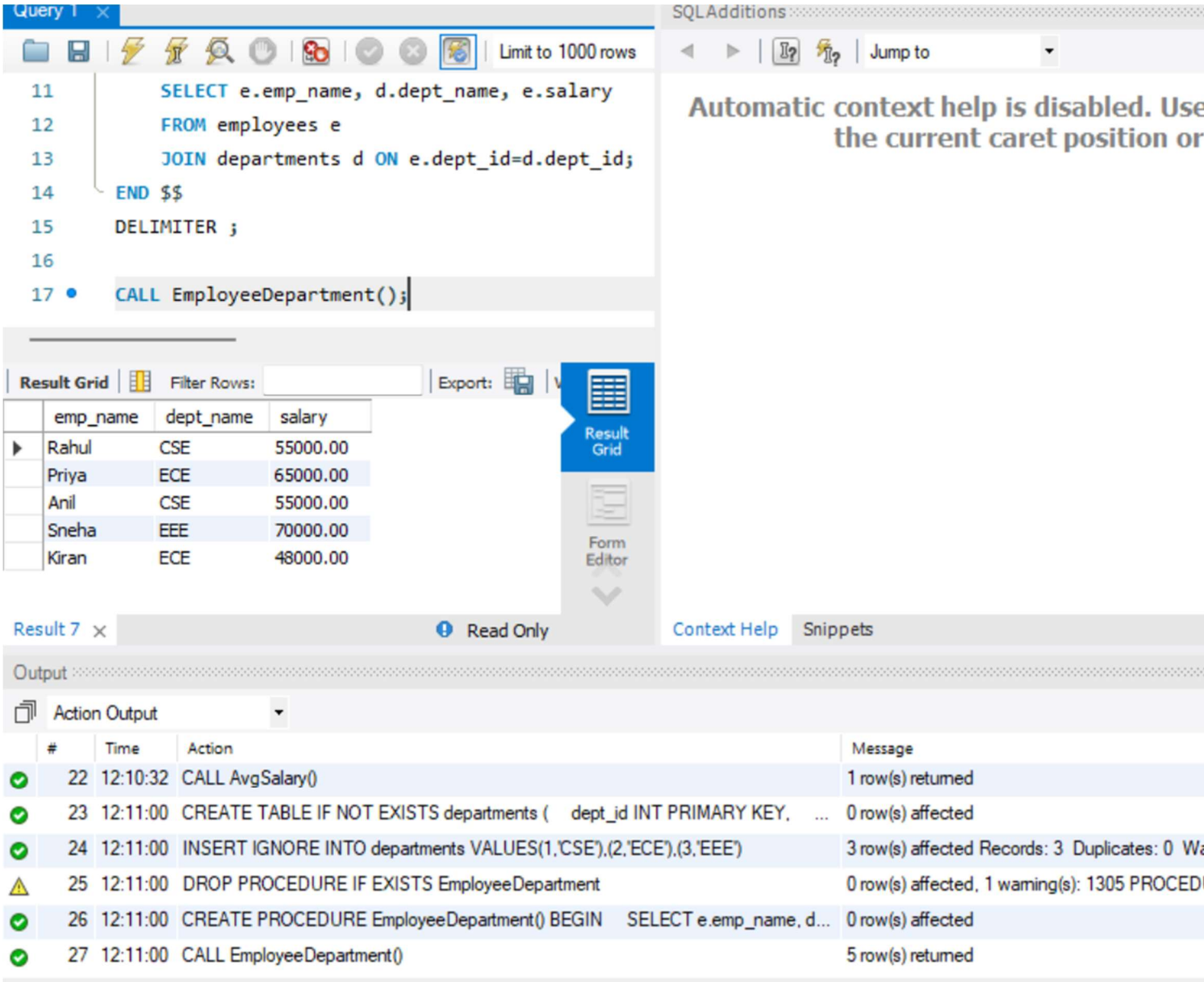


Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.

```
CREATE TABLE IF NOT EXISTS departments (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50)
);

INSERT IGNORE INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'EEE');
```

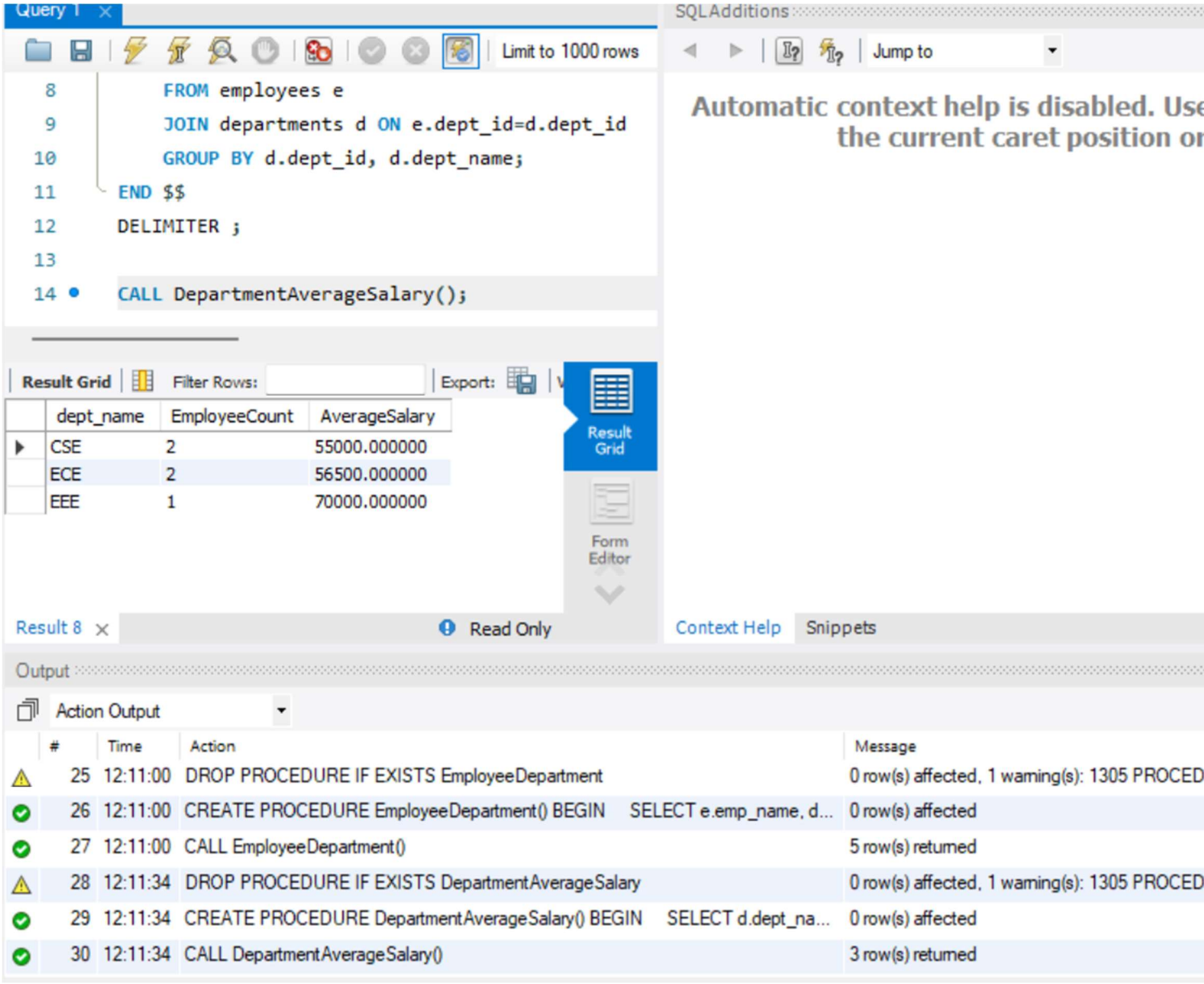
```
DROP PROCEDURE IF EXISTS EmployeeDepartment;
DELIMITER $$

CREATE PROCEDURE EmployeeDepartment()
BEGIN
   SELECT e.emp_name, d.dept_name, e.salary
   FROM employees e
   JOIN departments d ON e.dept_id=d.dept_id;
END $$

DELIMITER ;

CALL EmployeeDepartment();
```



Procedure 6 – GROUP BY and AVG

Question: Create a procedure to display department-wise employee count and average salary.

```
DROP PROCEDURE IF EXISTS DepartmentAverageSalary;
DELIMITER $$

CREATE PROCEDURE DepartmentAverageSalary()
BEGIN
   SELECT d.dept_name,
        COUNT(e.emp_id) AS EmployeeCount,
        AVG(e.salary) AS AverageSalary
   FROM employees e
   JOIN departments d ON e.dept_id=d.dept_id
   GROUP BY d.dept_id, d.dept_name;
END $$

DELIMITER ;

CALL DepartmentAverageSalary();
```

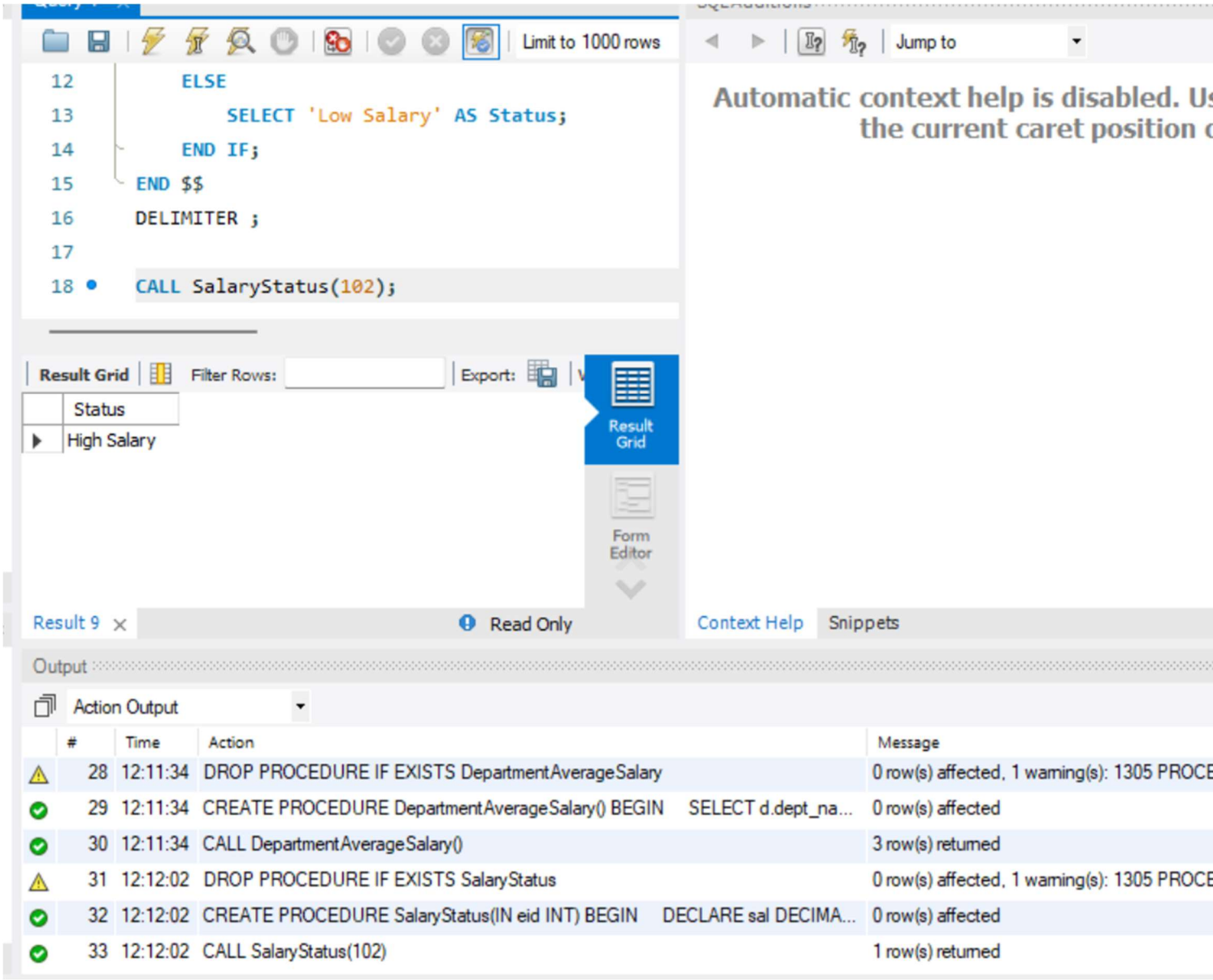


Procedure 7 – IF...ELSE

Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.

```
DROP PROCEDURE IF EXISTS SalaryStatus;
DELIMITER $$

CREATE PROCEDURE SalaryStatus(IN eid INT)
BEGIN
   DECLARE sal DECIMAL(10,2);

   SELECT salary INTO sal
   FROM employees
   WHERE emp_id=eid;
```

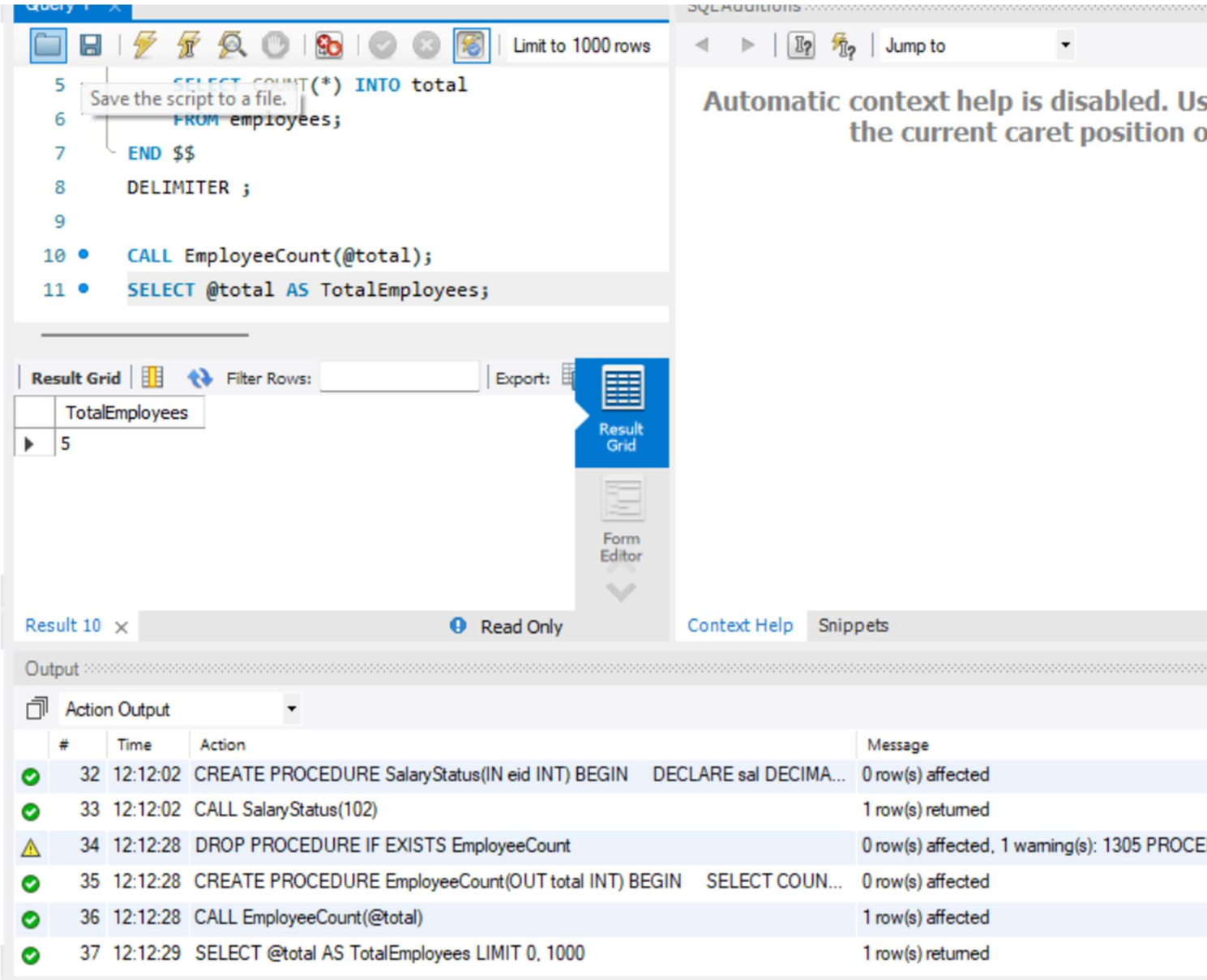
```
    IF sal >= 60000 THEN
        SELECT 'High Salary' AS Status;
    ELSE
        SELECT 'Low Salary' AS Status;
    END IF;
END $$

DELIMITER ;

CALL SalaryStatus(102);
```



Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.

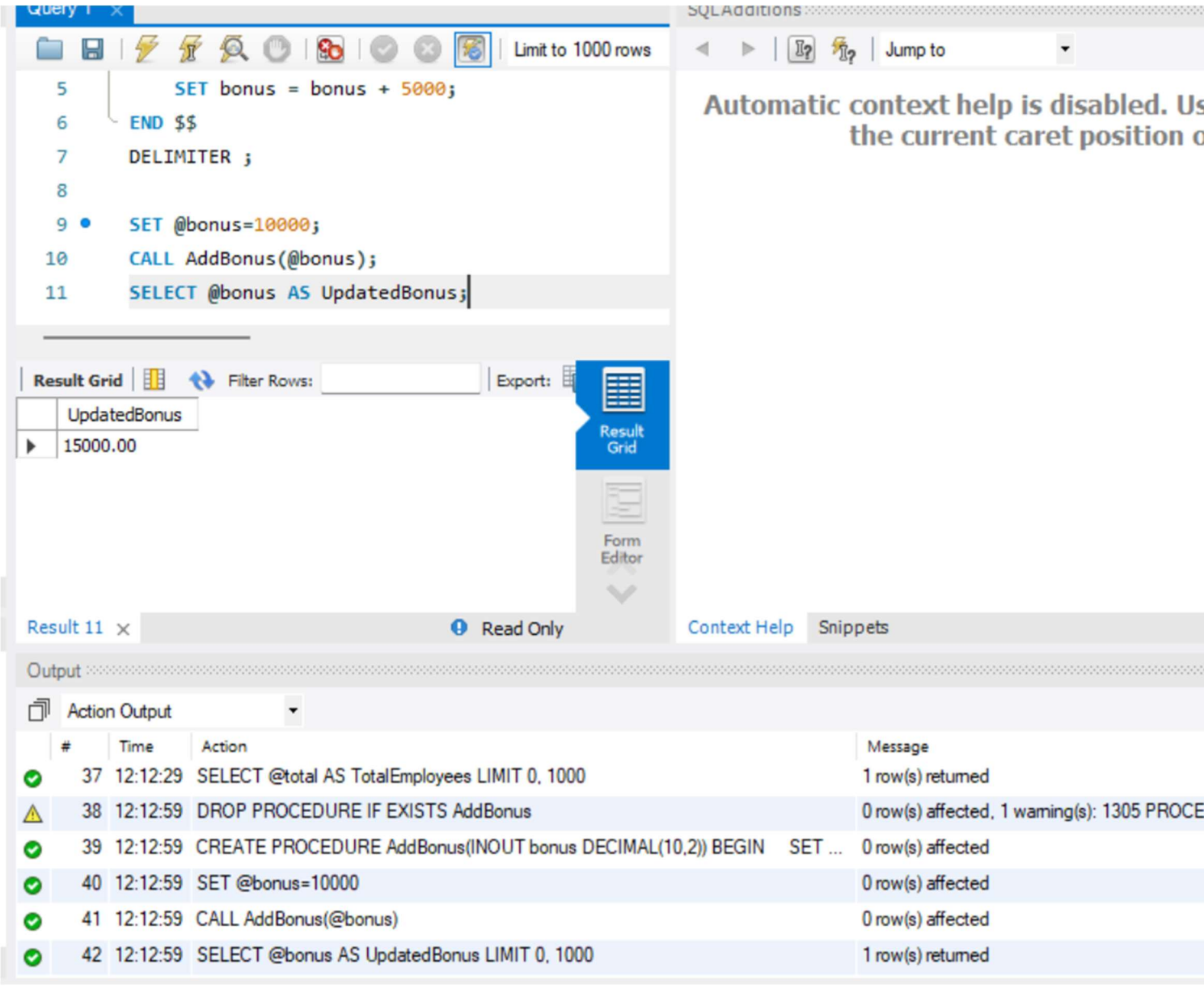
```
DROP PROCEDURE IF EXISTS EmployeeCount;
DELIMITER $$

CREATE PROCEDURE EmployeeCount(OUT total INT)
BEGIN
   SELECT COUNT(*) INTO total
   FROM employees;
END $$

DELIMITER ;

CALL EmployeeCount(@total);
SELECT @total AS TotalEmployees;
```



Procedure 9 – INOUT Parameter

Question: Receive a bonus through an INOUT parameter and add 5000 to it.

```
DROP PROCEDURE IF EXISTS AddBonus;
DELIMITER $$

CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
BEGIN
   SET bonus = bonus + 5000;
END $$

DELIMITER ;

SET @bonus=10000;
CALL AddBonus(@bonus);
SELECT @bonus AS UpdatedBonus;
```



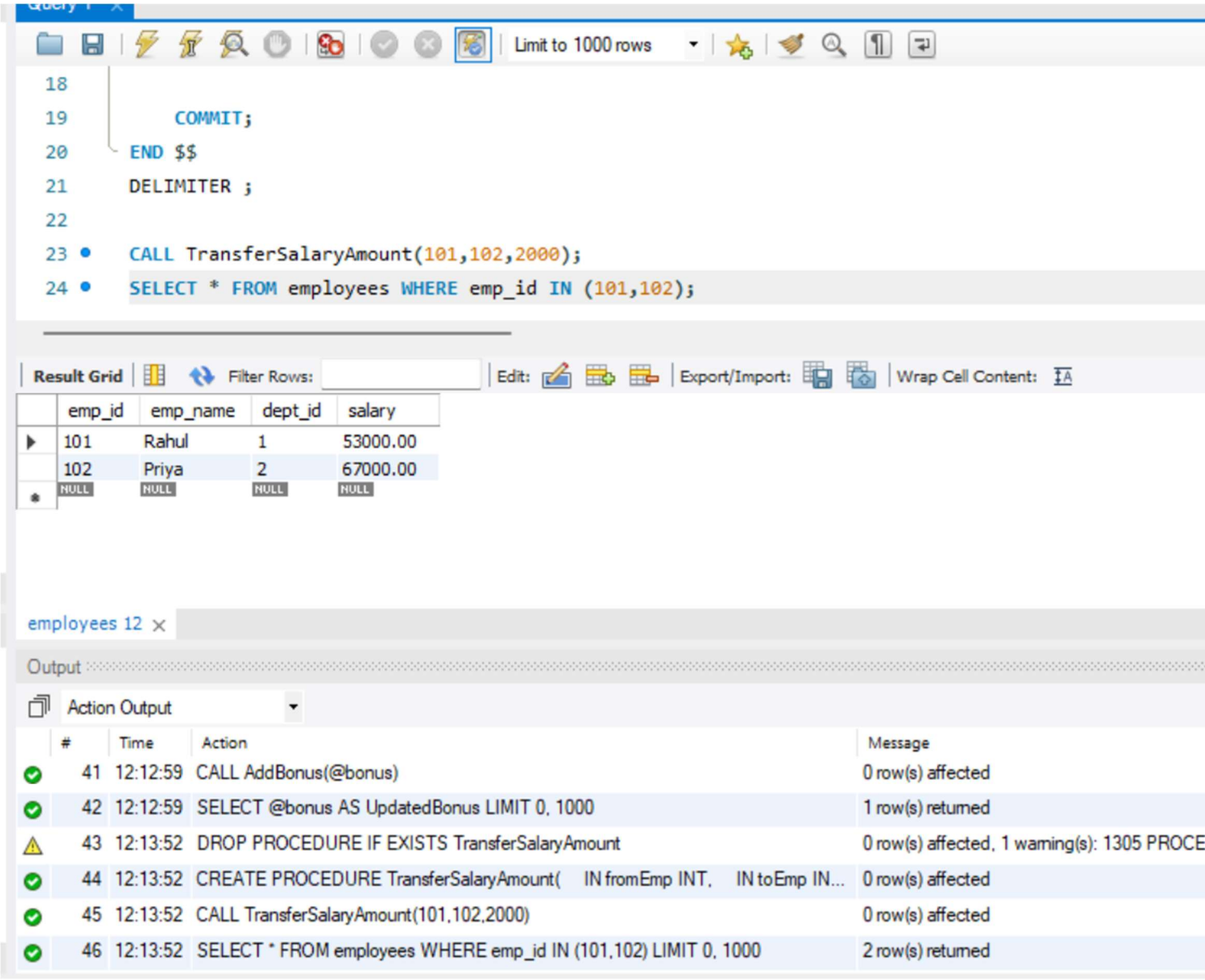
Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.

```
DROP PROCEDURE IF EXISTS TransferSalaryAmount;
DELIMITER $$

CREATE PROCEDURE TransferSalaryAmount(
   IN fromEmp INT,
   IN toEmp INT,
   IN amount DECIMAL(10,2)
)
BEGIN
```

```
    START TRANSACTION;

    UPDATE employees
    SET salary=salary-amount
    WHERE emp_id=fromEmp;

    UPDATE employees
    SET salary=salary+amount
    WHERE emp_id=toEmp;

    COMMIT;
END $$

DELIMITER ;

CALL TransferSalaryAmount(101,102,2000);
```

SELECT * FROM employees WHERE emp_id IN (101,102);

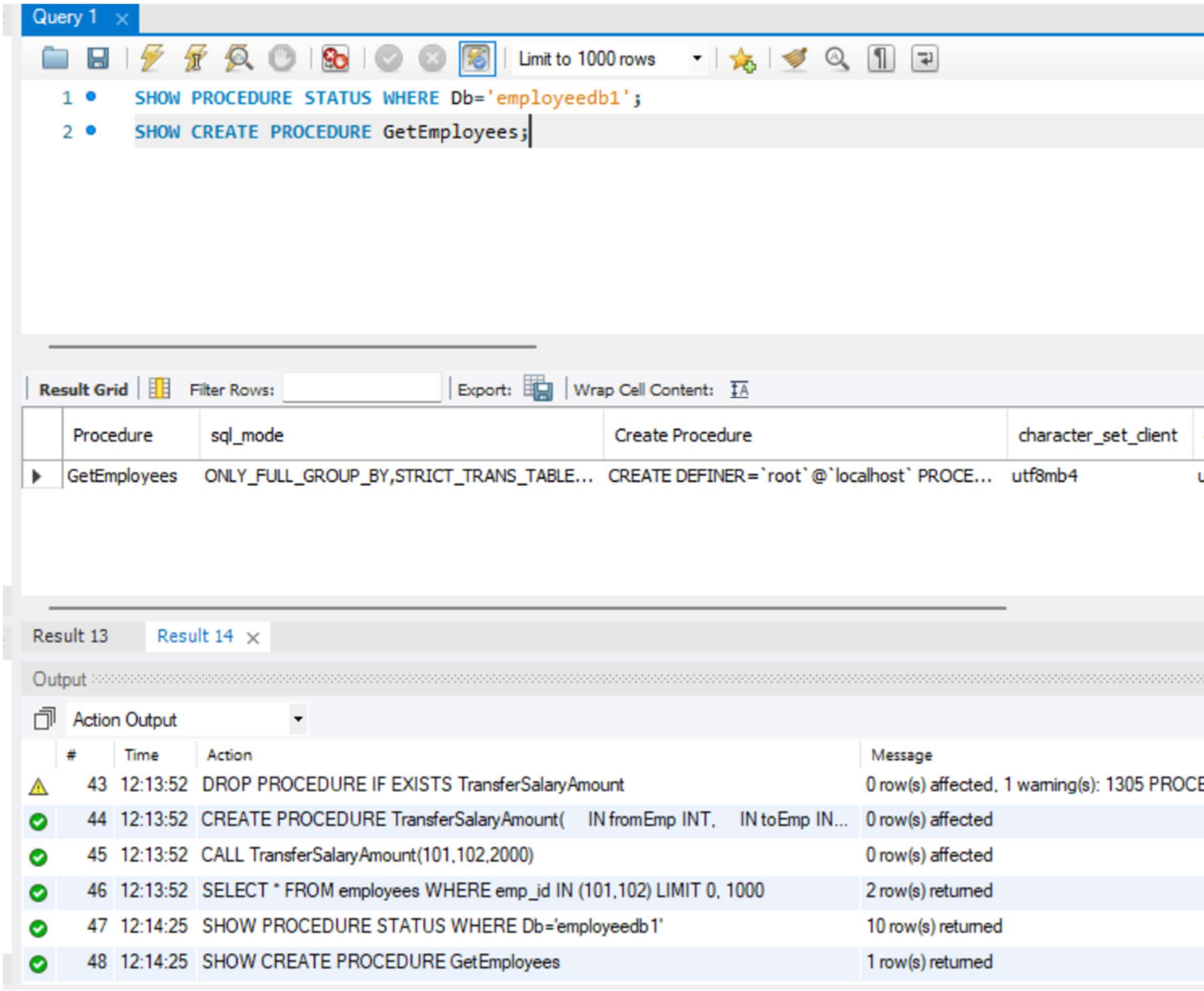


4. Useful Procedure Commands

SHOW PROCEDURE STATUS
WHERE Db='employeedb1';

SHOW CREATE PROCEDURE GetEmployees;

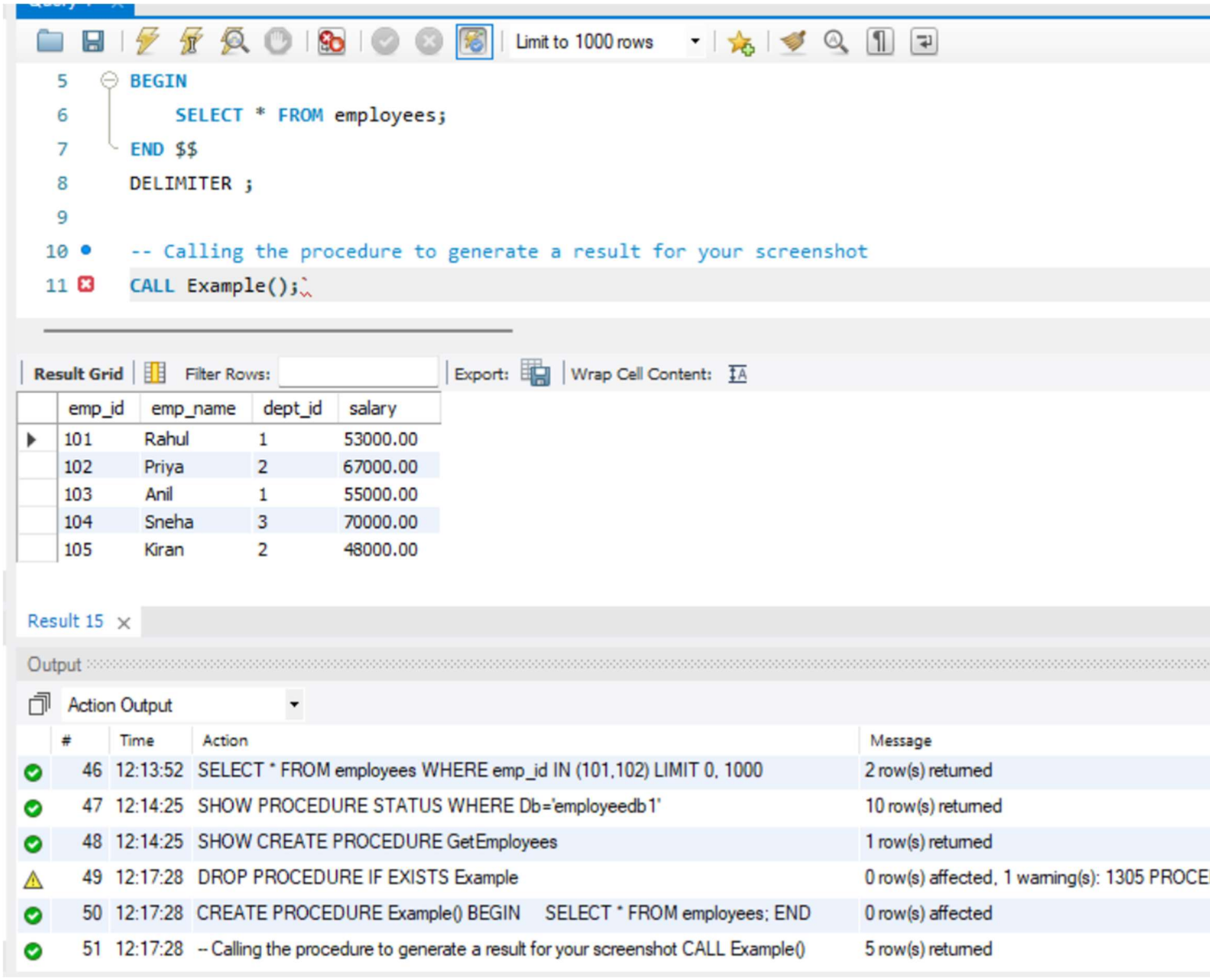
DROP PROCEDURE IF EXISTS GetEmployees;



5. Why DELIMITER is Used

DELIMITER temporarily changes the statement terminator in MySQL Workbench. It prevents the semicolons inside BEGIN...END from being interpreted as the end of the entire CREATE PROCEDURE statement.

```
DELIMITER $$
CREATE PROCEDURE Example()
BEGIN
   SELECT * FROM employees;
END $$
DELIMITER ;
```



```
5   BEGIN
6       SELECT * FROM employees;
7   END $$
8   DELIMITER ;
9
10  -- Calling the procedure to generate a result for your screenshot
11  CALL Example();
```

Limit to 1000 rows

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

Result 15

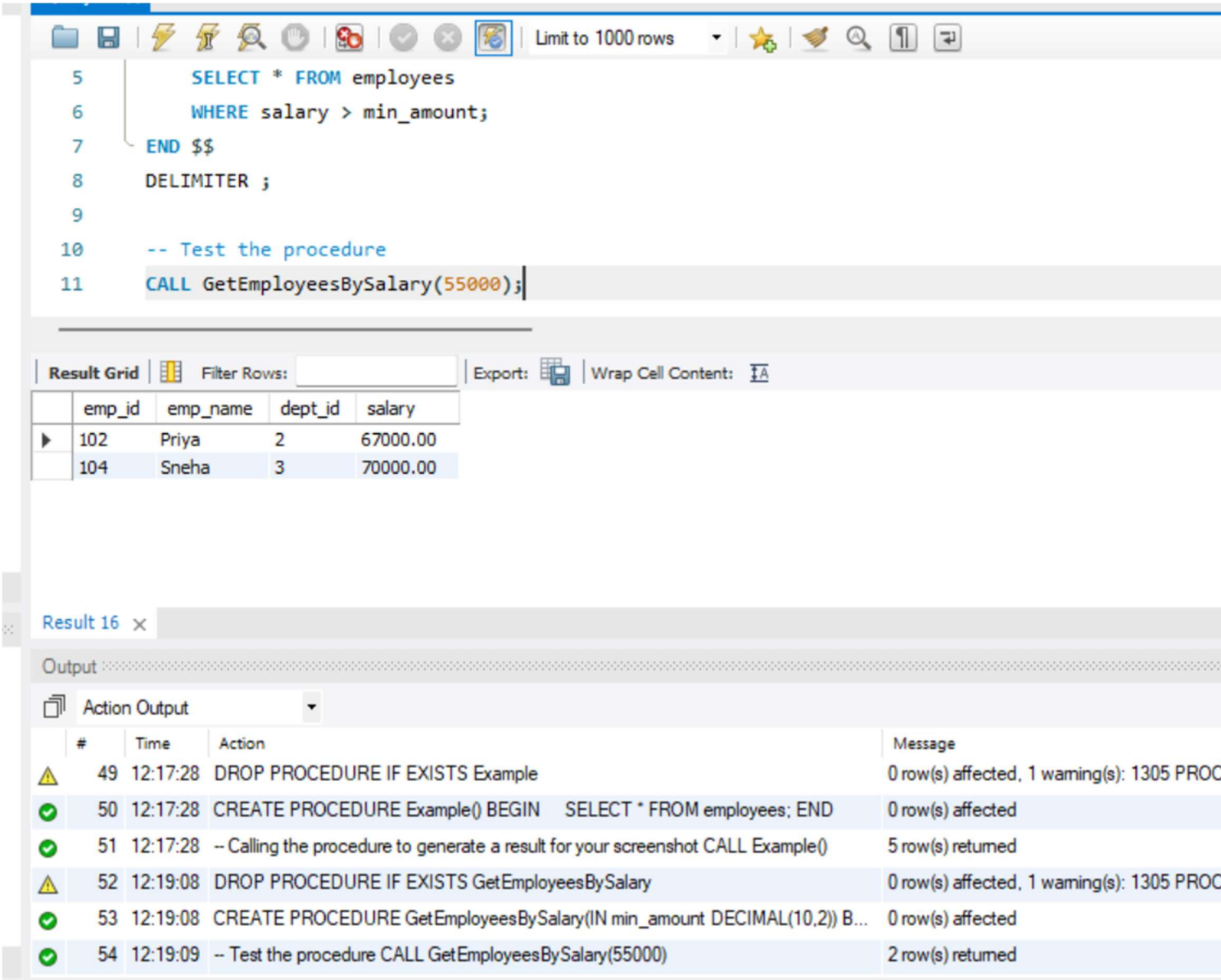
Output

Action Output

#	Time	Action	Message
46	12:13:52	SELECT * FROM employees WHERE emp_id IN (101,102) LIMIT 0, 1000	2 row(s) returned
47	12:14:25	SHOW PROCEDURE STATUS WHERE Db='employeedb1'	10 row(s) returned
48	12:14:25	SHOW CREATE PROCEDURE GetEmployees	1 row(s) returned
49	12:17:28	DROP PROCEDURE IF EXISTS Example	0 row(s) affected, 1 warning(s): 1305 PROCE
50	12:17:28	CREATE PROCEDURE Example() BEGIN SELECT * FROM employees; END	0 row(s) affected
51	12:17:28	-- Calling the procedure to generate a result for your screenshot CALL Example()	5 row(s) returned

6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.



```
    SELECT * FROM employees
    WHERE salary > min_amount;
END $$
DELIMITER ;

-- Test the procedure
CALL GetEmployeesBySalary(55000);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept_id	salary
102	Priya	2	67000.00
104	Sneha	3	70000.00

Result 16

Output

Action Output

#	Time	Action	Message
49	12:17:28	DROP PROCEDURE IF EXISTS Example	0 row(s) affected, 1 warning(s): 1305 PROC
50	12:17:28	CREATE PROCEDURE Example() BEGIN SELECT * FROM employees; END	0 row(s) affected
51	12:17:28	-- Calling the procedure to generate a result for your screenshot CALL Example()	5 row(s) returned
52	12:19:08	DROP PROCEDURE IF EXISTS GetEmployeesBySalary	0 row(s) affected, 1 warning(s): 1305 PROC
53	12:19:08	CREATE PROCEDURE GetEmployeesBySalary(IN min_amount DECIMAL(10,2)) B...	0 row(s) affected
54	12:19:09	-- Test the procedure CALL GetEmployeesBySalary(55000)	2 row(s) returned

2. Create a procedure to update an employee's department using IN parameters.

```
7       WHERE emp_id = e_id;
8   END $$
9   DELIMITER ;
10
11  -- Test the procedure by moving Anil (103) to dept 2
12  CALL UpdateEmployeeDept(103, 2);
13  SELECT * FROM employees WHERE emp_id = 103;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	dept_id	salary
103	Anil	2	55000.00
NULL	NULL	NULL	NULL

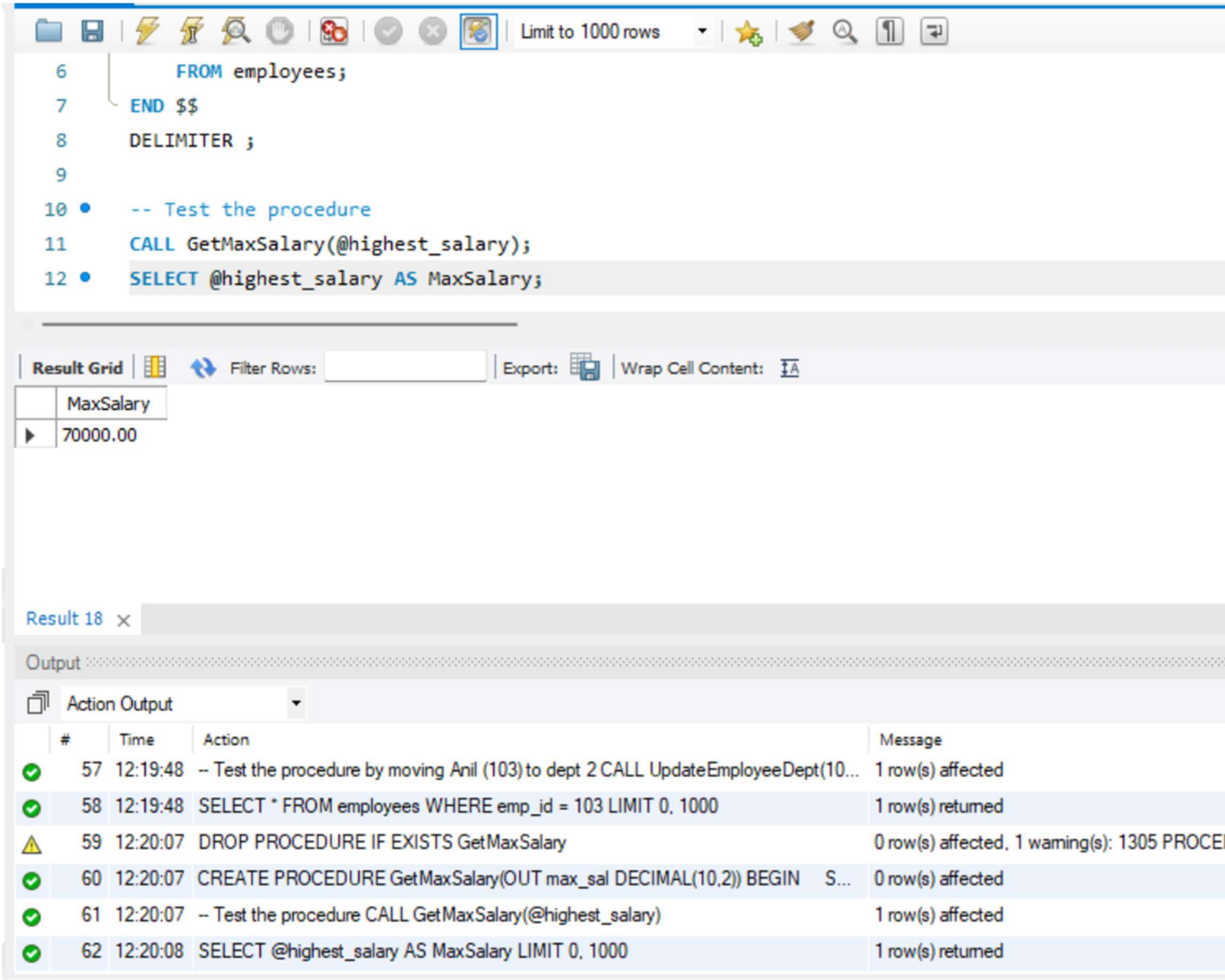
employees 17

Output

Action Output

#	Time	Action	Message
53	12:19:08	CREATE PROCEDURE GetEmployeesBySalary(IN min_amount DECIMAL(10,2)) B...	0 row(s) affected
54	12:19:09	-- Test the procedure CALL GetEmployeesBySalary(55000)	2 row(s) returned
55	12:19:48	DROP PROCEDURE IF EXISTS UpdateEmployeeDept	0 row(s) affected, 1 warning(s): 1305 PROCEDU
56	12:19:48	CREATE PROCEDURE UpdateEmployeeDept(IN e_id INT, IN new_d_id INT) BEG...	0 row(s) affected
57	12:19:48	-- Test the procedure by moving Anil (103) to dept 2 CALL UpdateEmployeeDept(10...	1 row(s) affected
58	12:19:48	SELECT * FROM employees WHERE emp_id = 103 LIMIT 0, 1000	1 row(s) returned

3. Create a procedure to return the maximum salary using an OUT parameter.



```
6       FROM employees;
7   END $$
8   DELIMITER ;
9
10  -- Test the procedure
11  CALL GetMaxSalary(@highest_salary);
12  SELECT @highest_salary AS MaxSalary;
```

Limit to 1000 rows

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

MaxSalary
70000.00

Result 18

Output

Action Output

#	Time	Action	Message
57	12:19:48	-- Test the procedure by moving Anil (103) to dept 2 CALL UpdateEmployeeDept(10...	1 row(s) affected
58	12:19:48	SELECT * FROM employees WHERE emp_id = 103 LIMIT 0, 1000	1 row(s) returned
59	12:20:07	DROP PROCEDURE IF EXISTS GetMaxSalary	0 row(s) affected, 1 warning(s): 1305 PROCE
60	12:20:07	CREATE PROCEDURE GetMaxSalary(OUT max_sal DECIMAL(10,2)) BEGIN S...	0 row(s) affected
61	12:20:07	-- Test the procedure CALL GetMaxSalary(@highest_salary)	1 row(s) affected
62	12:20:08	SELECT @highest_salary AS MaxSalary LIMIT 0, 1000	1 row(s) returned

4. Create a procedure to return the employee count of a specified department.

```
    FROM employees
    WHERE dept_id = d_id;
END $$
DELIMITER ;

-- Test the procedure for department 2
CALL GetDeptEmployeeCount(2);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeCount
3

Result 19

Output

Action Output

#	Time	Action	Message
60	12:20:07	CREATE PROCEDURE GetMaxSalary(OUT max_sal DECIMAL(10,2)) BEGIN S...	0 row(s) affected
61	12:20:07	-- Test the procedure CALL GetMaxSalary(@highest_salary)	1 row(s) affected
62	12:20:08	SELECT @highest_salary AS MaxSalary LIMIT 0, 1000	1 row(s) returned
63	12:20:35	DROP PROCEDURE IF EXISTS GetDeptEmployeeCount	0 row(s) affected, 1 warning(s): 1305 PROCED
64	12:20:35	CREATE PROCEDURE GetDeptEmployeeCount(IN d_id INT) BEGIN SELECT C...	0 row(s) affected
65	12:20:35	-- Test the procedure for department 2 CALL GetDeptEmployeeCount(2)	1 row(s) returned

5. Create a procedure using JOIN to display employee and department details.

```
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id;
END $$
DELIMITER ;

-- Test the procedure
CALL EmployeeDeptDetails();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept_name	salary
101	Rahul	CSE	53000.00
102	Priya	ECE	67000.00
103	Anil	ECE	55000.00
104	Sneha	EEE	70000.00
105	Kiran	ECE	48000.00

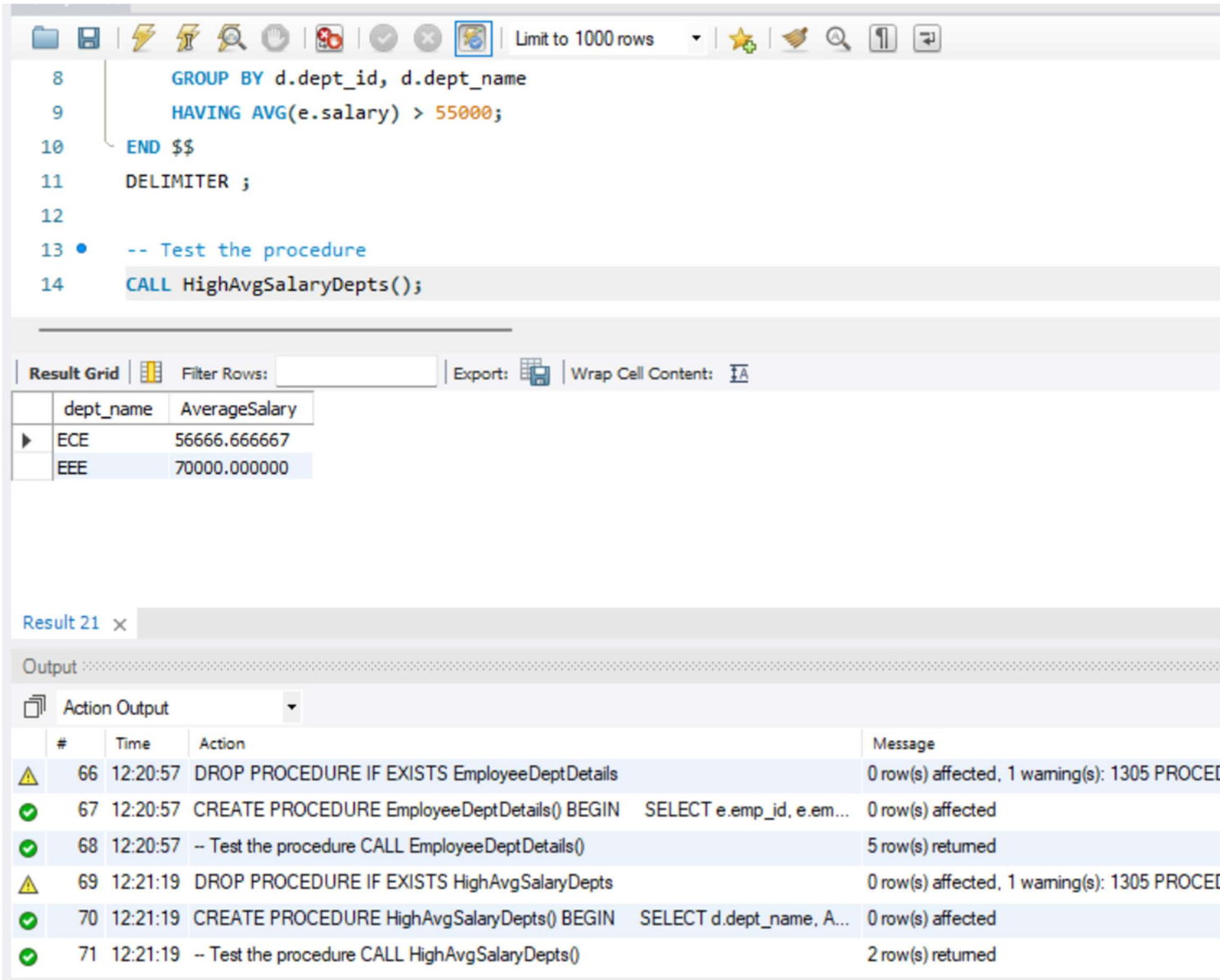
Result 20

Output

Action Output

#	Time	Action	Message
63	12:20:35	DROP PROCEDURE IF EXISTS GetDeptEmployeeCount	0 row(s) affected, 1 warning(s): 1305 PROCEDU
64	12:20:35	CREATE PROCEDURE GetDeptEmployeeCount(IN d_id INT) BEGIN SELECT C...	0 row(s) affected
65	12:20:35	-- Test the procedure for department 2 CALL GetDeptEmployeeCount(2)	1 row(s) returned
66	12:20:57	DROP PROCEDURE IF EXISTS EmployeeDeptDetails	0 row(s) affected, 1 warning(s): 1305 PROCEDU
67	12:20:57	CREATE PROCEDURE EmployeeDeptDetails() BEGIN SELECT e.emp_id, e.em...	0 row(s) affected
68	12:20:57	-- Test the procedure CALL EmployeeDeptDetails()	5 row(s) returned

6. Create a procedure to display departments whose average salary is greater than 55000.



```
8       GROUP BY d.dept_id, d.dept_name
9       HAVING AVG(e.salary) > 55000;
10  END $$
11  DELIMITER ;
12
13  -- Test the procedure
14  CALL HighAvgSalaryDepts();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	AverageSalary
ECE	56666.666667
EEE	70000.000000

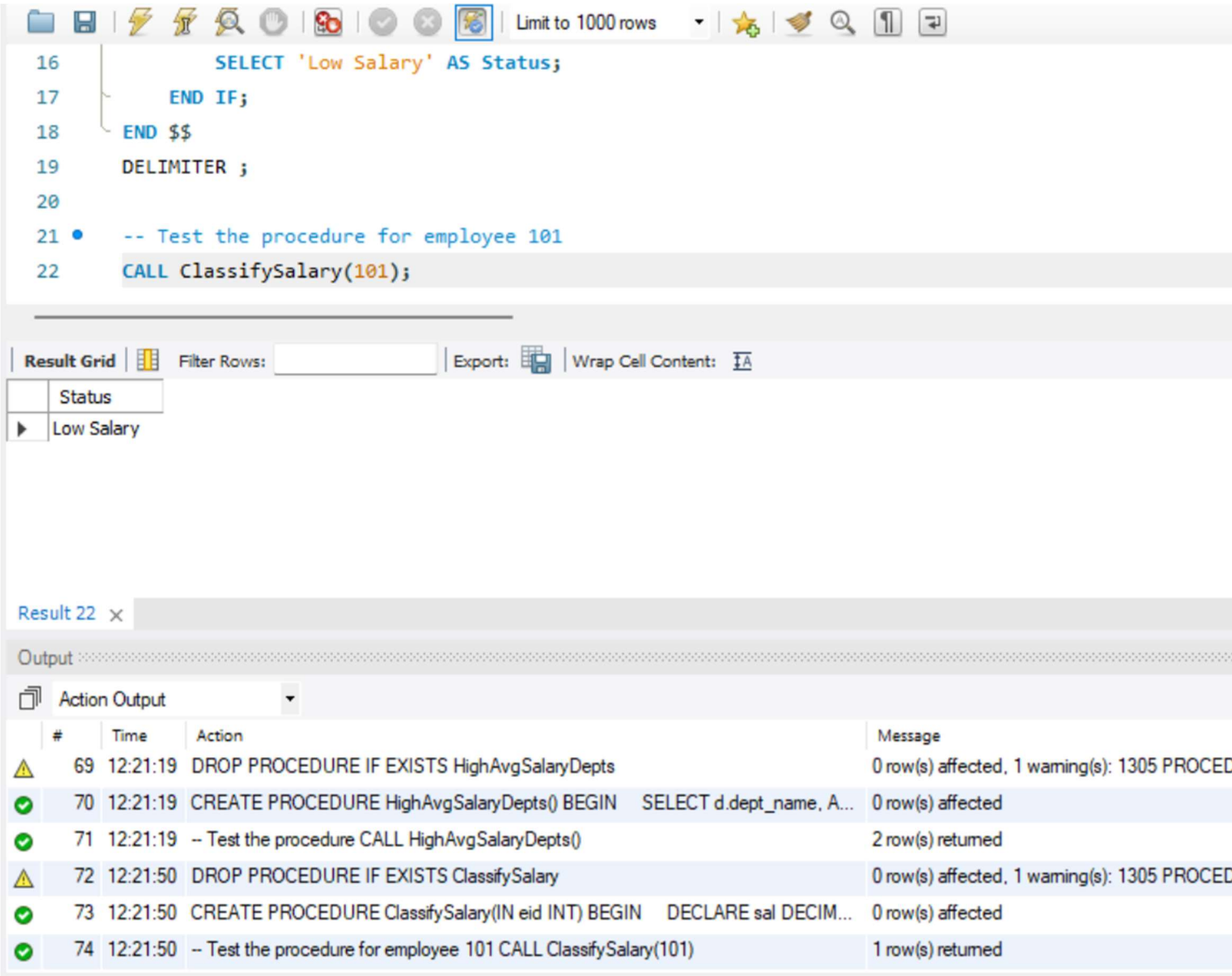
Result 21

Output

Action Output

#	Time	Action	Message
66	12:20:57	DROP PROCEDURE IF EXISTS EmployeeDeptDetails	0 row(s) affected, 1 warning(s): 1305 PROCED
67	12:20:57	CREATE PROCEDURE EmployeeDeptDetails() BEGIN SELECT e.emp_id, e.em...	0 row(s) affected
68	12:20:57	-- Test the procedure CALL EmployeeDeptDetails()	5 row(s) returned
69	12:21:19	DROP PROCEDURE IF EXISTS HighAvgSalaryDepts	0 row(s) affected, 1 warning(s): 1305 PROCED
70	12:21:19	CREATE PROCEDURE HighAvgSalaryDepts() BEGIN SELECT d.dept_name, A...	0 row(s) affected
71	12:21:19	-- Test the procedure CALL HighAvgSalaryDepts()	2 row(s) returned

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.



```
16          SELECT 'Low Salary' AS Status;
17      END IF;
18  END $$
19  DELIMITER ;
20
21  -- Test the procedure for employee 101
22  CALL ClassifySalary(101);
```

Limit to 1000 rows

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Status
Low Salary

Result 22

Output

Action Output

#	Time	Action	Message
69	12:21:19	DROP PROCEDURE IF EXISTS HighAvgSalaryDepts	0 row(s) affected, 1 warning(s): 1305 PROCED
70	12:21:19	CREATE PROCEDURE HighAvgSalaryDepts() BEGIN SELECT d.dept_name, A...	0 row(s) affected
71	12:21:19	-- Test the procedure CALL HighAvgSalaryDepts()	2 row(s) returned
72	12:21:50	DROP PROCEDURE IF EXISTS ClassifySalary	0 row(s) affected, 1 warning(s): 1305 PROCED
73	12:21:50	CREATE PROCEDURE ClassifySalary(IN eid INT) BEGIN DECLARE sal DECIM...	0 row(s) affected
74	12:21:50	-- Test the procedure for employee 101 CALL ClassifySalary(101)	1 row(s) returned

8. Create a procedure that accepts a department ID and returns its average salary.

```
    FROM employees
    WHERE dept_id = d_id;
END $$
DELIMITER ;

-- Test the procedure for department 1
CALL GetAvgSalaryByDept(1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

AverageSalary
53000.000000

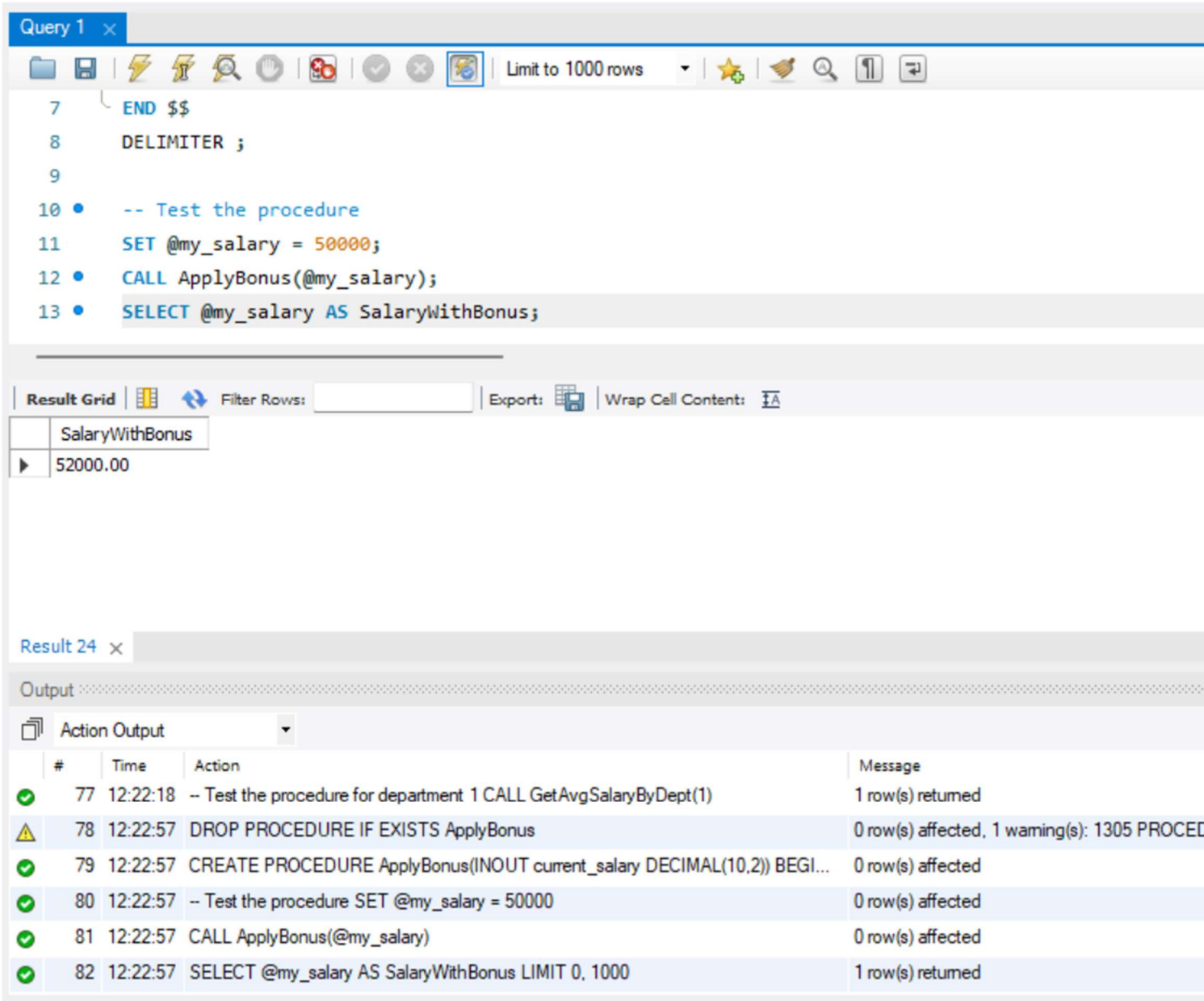
Result 23

Output

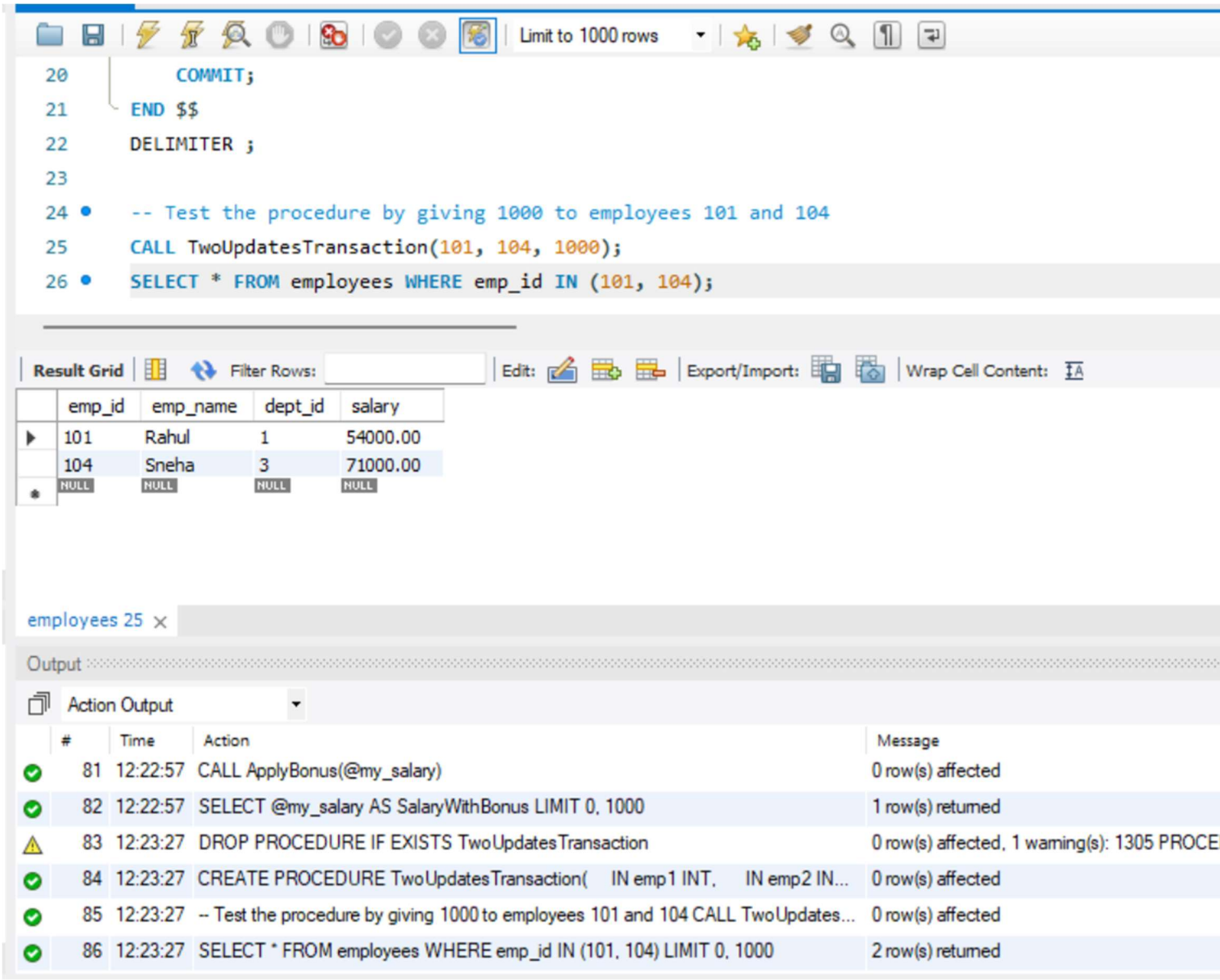
Action Output

#	Time	Action	Message
72	12:21:50	DROP PROCEDURE IF EXISTS ClassifySalary	0 row(s) affected, 1 warning(s): 1305 PROCED
73	12:21:50	CREATE PROCEDURE ClassifySalary(IN eid INT) BEGIN DECLARE sal DECIM...	0 row(s) affected
74	12:21:50	-- Test the procedure for employee 101 CALL ClassifySalary(101)	1 row(s) returned
75	12:22:18	DROP PROCEDURE IF EXISTS GetAvgSalaryByDept	0 row(s) affected, 1 warning(s): 1305 PROCED
76	12:22:18	CREATE PROCEDURE GetAvgSalaryByDept(IN d_id INT) BEGIN SELECT AV...	0 row(s) affected
77	12:22:18	-- Test the procedure for department 1 CALL GetAvgSalaryByDept(1)	1 row(s) returned

9. Create a procedure using INOUT to apply a bonus to an input salary.



10. Create a procedure that performs two related updates inside a transaction.



```
20      COMMIT;
21  END $$
22  DELIMITER ;
23
24  -- Test the procedure by giving 1000 to employees 101 and 104
25  CALL TwoUpdatesTransaction(101, 104, 1000);
26  SELECT * FROM employees WHERE emp_id IN (101, 104);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

emp_id	emp_name	dept_id	salary
101	Rahul	1	54000.00
104	Sneha	3	71000.00
NULL	NULL	NULL	NULL

employees 25

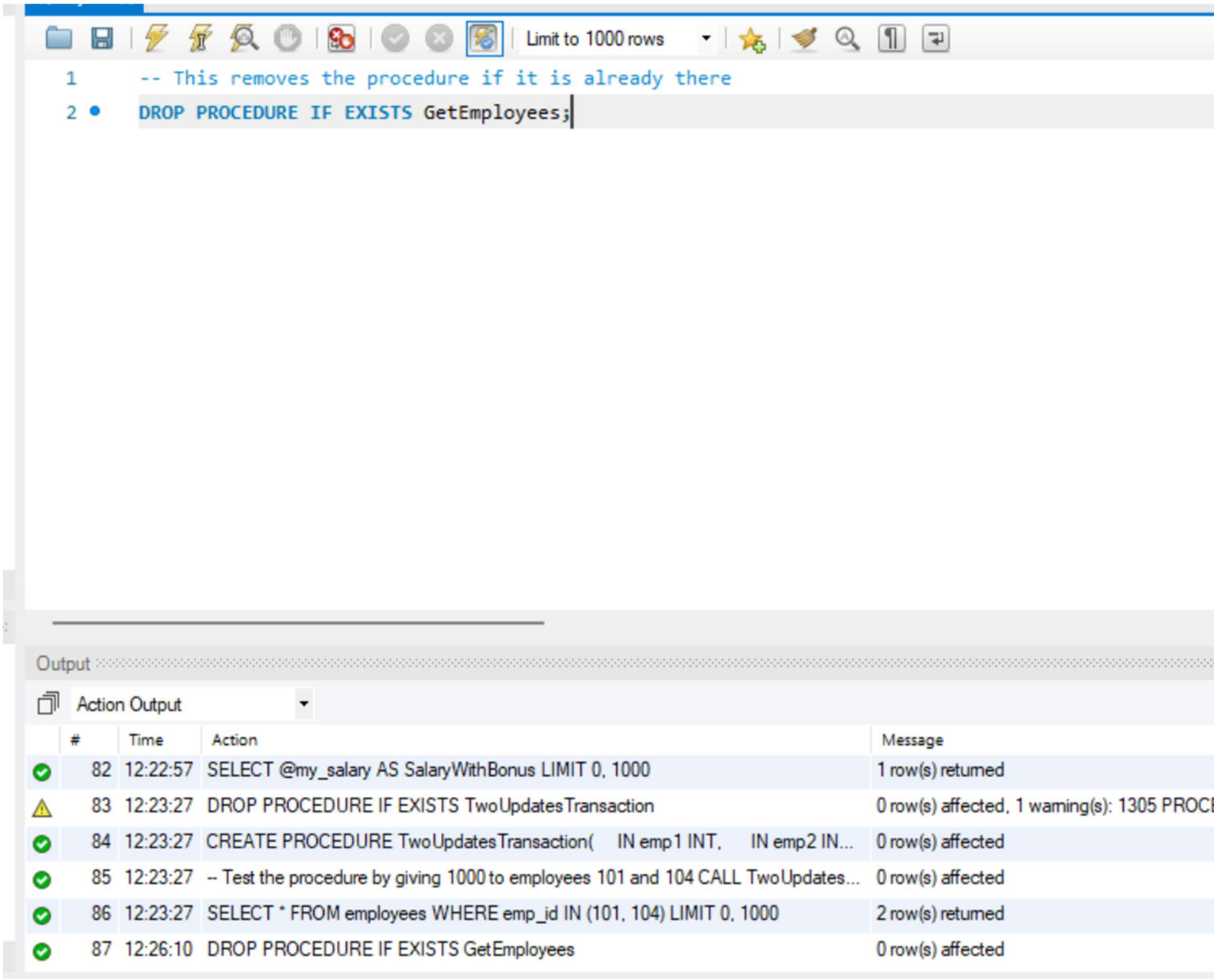
Output

Action Output

#	Time	Action	Message
81	12:22:57	CALL ApplyBonus(@my_salary)	0 row(s) affected
82	12:22:57	SELECT @my_salary AS SalaryWithBonus LIMIT 0, 1000	1 row(s) returned
83	12:23:27	DROP PROCEDURE IF EXISTS TwoUpdatesTransaction	0 row(s) affected, 1 warning(s): 1305 PROCE
84	12:23:27	CREATE PROCEDURE TwoUpdatesTransaction(IN emp1 INT, IN emp2 IN...	0 row(s) affected
85	12:23:27	-- Test the procedure by giving 1000 to employees 101 and 104 CALL TwoUpdates...	0 row(s) affected
86	12:23:27	SELECT * FROM employees WHERE emp_id IN (101, 104) LIMIT 0, 1000	2 row(s) returned

7. Common Errors

Procedure already exists: Use DROP PROCEDURE IF EXISTS before recreating it.



Syntax error near BEGIN: Check DELIMITER $$ and DELIMITER ;.

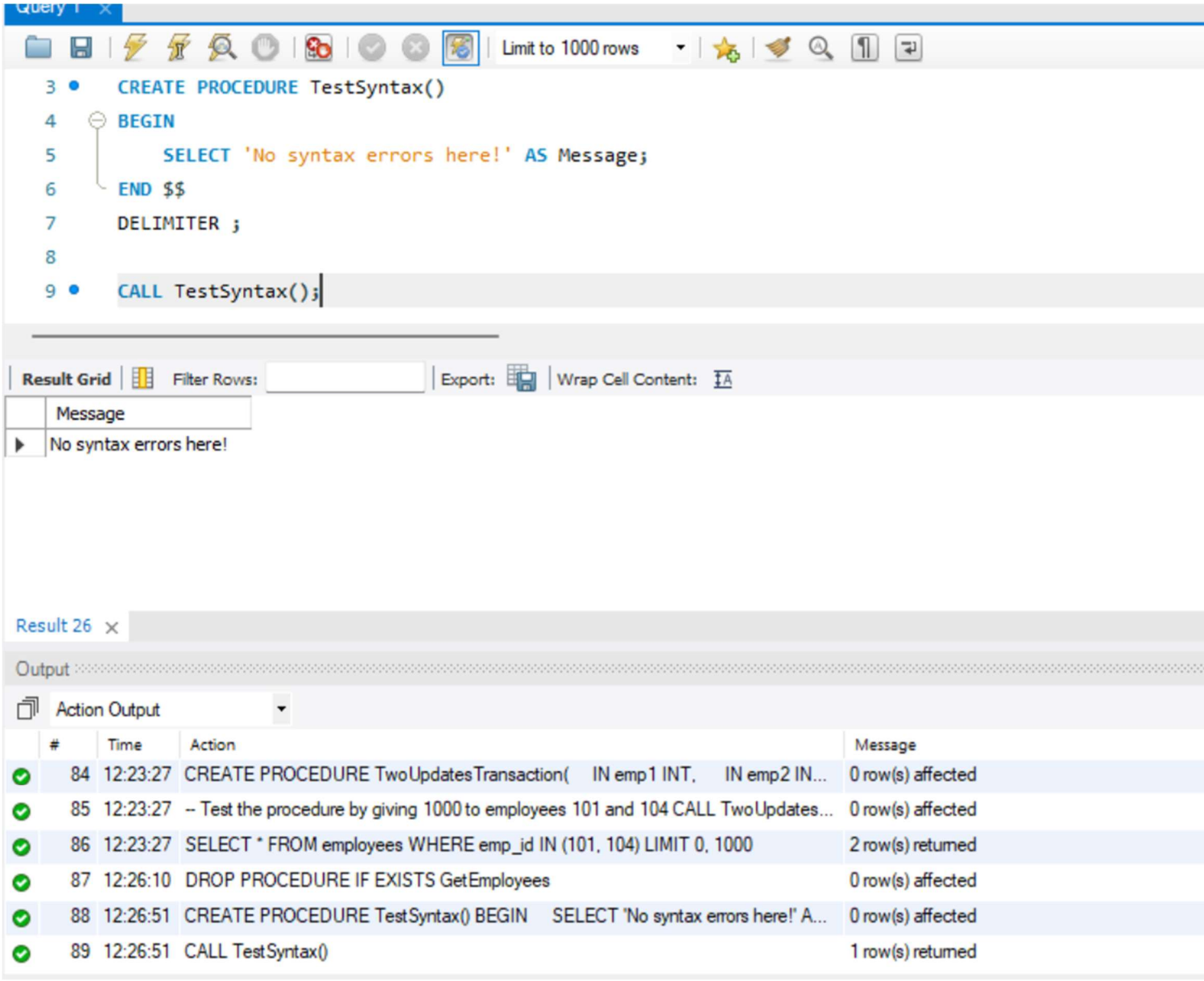
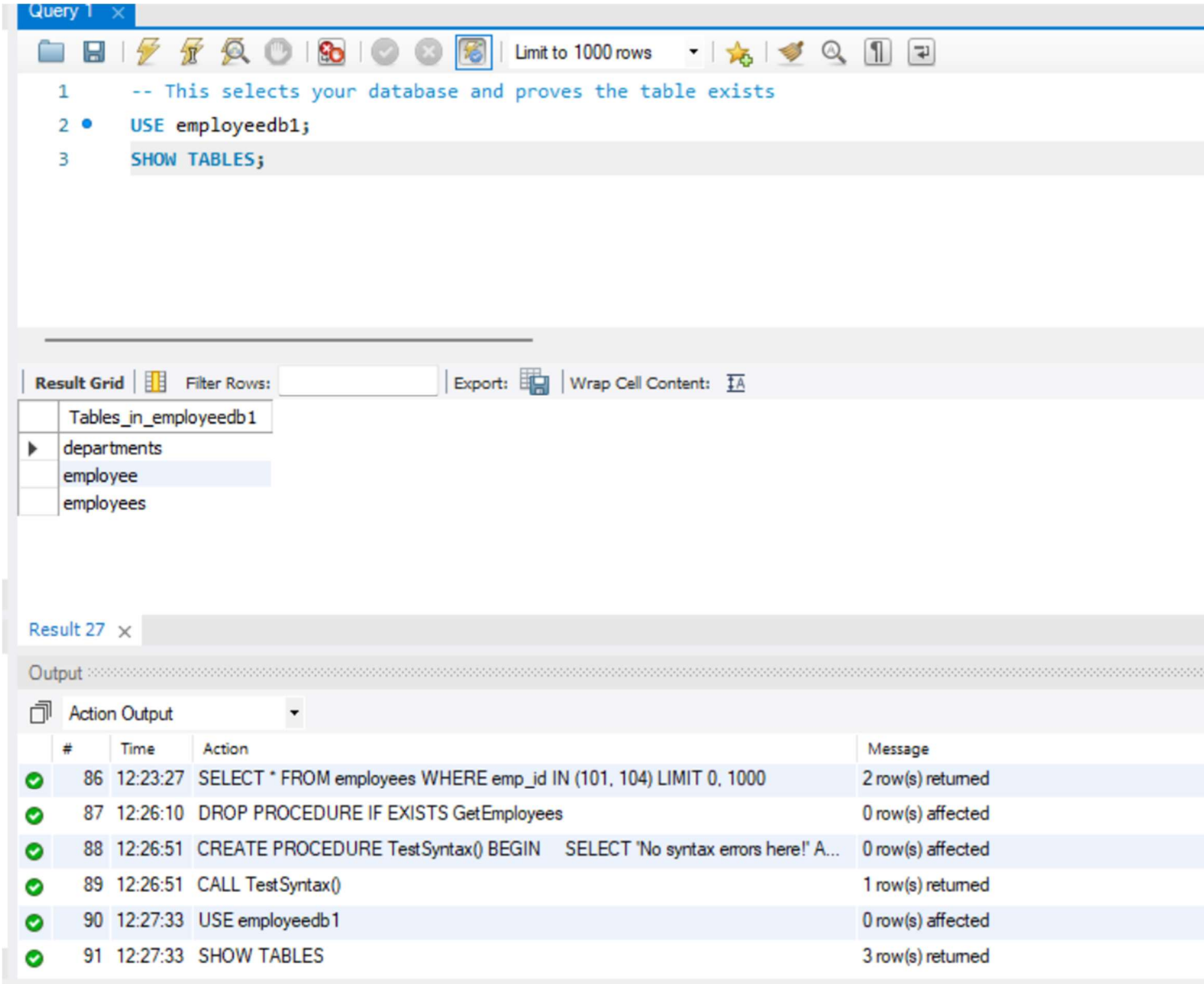


Table does not exist: Run USE employeedb1; and SHOW TABLES;.



Unknown column: Check names using DESCRIBE employees;